

Lecture–03: Binary Search

Definition

Binary Search is an efficient searching algorithm used to find an element in a **sorted list**. Instead of checking every element one by one, it repeatedly divides the search space into two halves until the required element is found or the search interval becomes empty.

Important Note

Prerequisite:

Binary Search works only on a **sorted array or list**. If the data is not sorted, it must be sorted before applying Binary Search.

Algorithm

Algorithm

1. Set two pointers:

$$low = 0, \quad high = n - 1$$

2. Repeat while

$$low \leq high$$

3. Compute the middle index:

$$mid = low + \frac{high - low}{2}$$

4. Compare the middle element with the target.

- If equal, return the index.
- If the target is greater, search the right half.
- If the target is smaller, search the left half.

5. If the loop ends, the element is not present in the list.

Example

Worked Example

Find the element **23** in the sorted list:

[2, 5, 8, 12, 16, 23, 38, 56]

Low	High	Mid	Value	Action
0	7	3	12	Search Right
4	7	5	23	Found

Python Implementation

Python Program: Binary Search

```
1 def binary_search(arr, target):
2     low = 0
3     high = len(arr) - 1
4
5     while low <= high:
6         mid = low + (high - low) // 2
7
8         if arr[mid] == target:
9             return mid
10
11        elif arr[mid] < target:
12            low = mid + 1
13
14        else:
15            high = mid - 1
16
17    return -1
18
19
20 numbers = [2, 5, 8, 12, 16, 23, 38, 56]
21 target = 23
22
23 index = binary_search(numbers, target)
24
25 if index != -1:
26     print("Element found at index", index)
27 else:
28     print("Element not found")
```

Complexity Analysis

Definition

Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$
Space Complexity	$O(1)$ (Iterative)

Advantages

- Very fast for large sorted datasets.
- Requires only logarithmic time.
- Uses constant extra memory in the iterative implementation.

Limitations

- Works only on sorted data.
- Not suitable for unsorted lists without sorting first.
- Random access to elements is required, making it ideal for arrays but less efficient for linked lists.

Summary

Important Note

Binary Search repeatedly divides the search interval into two halves. Since the search space is reduced by half after every comparison, the algorithm runs in $\mathbf{O}(\log \mathbf{n})$ time, making it much faster than Linear Search for large sorted datasets.